HATHOR-CORE · MASTER-BOOK

# Verification — Checking the Rules

How the node decides a single vertex is valid on its own — proof-of-work, signatures via script evaluation, value conservation, and the structural rules — before consensus ever weighs in.

| | |
|---|---|
| SUBJECT | hathor-core · Part II · the node, end to end |
| CHAPTER | 31 · Part II · The Node |
| BRANCH | study/hathor-core-studies |
| EDITION | 2026 · v1 |

Verification · VerificationService · verify_pow · Script evaluation · Stack machine · P2PKH · Signatures · Value conservation · Basic vs full verify

# Table of Contents

# Chapter 31 — Verification: Checking the Rules

**What you'll learn in this chapter**

- The single question this package answers: is this vertex valid on its own? — and why that is a different question from the one consensus (Ch 32) asks.
- The shape of `hathor/verification/`: one orchestrator (`VerificationService`) and a family of per-type verifiers, wired together by `VertexVerifiers`.
- The **basic-vs-full** split — why some checks can run with no other data on hand, and others need the rest of the ledger — and how that split maps onto a vertex's validation state.
- The concrete rules, each tied to code: proof-of-work, minimum weight, value conservation and token rules, structural parent/timestamp rules, sigops bounds, and the reward lock.
- **Script evaluation as a stack machine** — built up from a neutral toy first, then the real `P2PKH` script and the opcodes behind it. This is where Chapter 7's "ownership is a satisfiable lock" finally gets paid off in code.
- Where verification sits in the lifecycle: invoked by the vertex handler (Ch 33) on every new vertex, before consensus — and what "it failed verification" means for the ledger.

A full node never takes another node's word for anything. When a block or transaction arrives — downloaded during sync, broadcast by a peer, or pushed in through the API — the node's first job is to ask: does this thing obey the protocol's rules at all? Not "is it the version of history we'll keep" — that is consensus, the next chapter. The narrower, prior question is: is this vertex even **well-formed and rule-abiding**, considered by itself? That is what `hathor/verification/` decides.

This is a gate, and it is unforgiving. A vertex that fails verification is rejected outright. It is never written to storage as valid, never indexed, never shown to wallets, never given to consensus to weigh. The rest of the node gets to assume that anything it handles has already passed through here.

---

## 31.1 Localization

The package is small and flat. Each file holds one verifier class; one file holds the orchestrator that calls them in the right order.

```
hathor-core/
└── hathor/
    ├── verification/                          ◀ YOU ARE HERE
    │   ├── verification_service.py    ← VerificationService: the orchestrator
    │   ├── vertex_verifiers.py        ← VertexVerifiers: the bundle of all verifiers
    │   ├── verification_params.py     ← VerificationParams: per-run settings/flags
    │   ├── vertex_verifier.py         ← VertexVerifier: checks shared by ALL vertices
    │   ├── block_verifier.py          ← BlockVerifier: block-only rules
    │   ├── transaction_verifier.py    ← TransactionVerifier: tx-only rules
    │   ├── merge_mined_block_verifier.py  ← aux-pow (merged mining)
    │   ├── poa_block_verifier.py      ← proof-of-authority block rules
    │   ├── token_creation_transaction_verifier.py
    │   ├── nano_header_verifier.py    ← nano-contract header rules (→ Ch 39)
    │   ├── fee_header_verifier.py     ← fee-header rules
    │   └── on_chain_blueprint_verifier.py ← on-chain blueprint rules (→ Ch 39)
    │
    └── transaction/
        └── scripts/                   ← the script "virtual machine" lives here, not in veri
            ├── execute.py             ← script_eval / execute_eval: the evaluator
            ├── opcode.py              ← the Opcode enum + one function per opcode
            ├── p2pkh.py               ← the standard pay-to-public-key-hash script
            ├── multi_sig.py           ← the m-of-n multisignature script
            └── construct.py           ← helpers to build/parse scripts
```

One thing to fix in mind early: the **script evaluator does not live in `verification/`**. It lives in `hathor/transaction/scripts/`, alongside the vertex model. Verification calls into it — when the transaction verifier needs to check that an input is allowed to spend an output, it hands the two scripts to `script_eval` (`transaction/scripts/execute.py:103`). We treat the script machine here, in this chapter, because checking signatures is a verification rule; but the code that implements it is shared with wallets and miners, so it sits in the model package, not the rule package.

> **Context.** Verification is the "is this even valid?" stage of the ingestion pipeline you met in §0.3. Every new vertex flows: **verify** (this chapter) → **consensus** (Ch 32, which decides canonical history) → **store + index** (Ch 27–30). The component that runs this pipeline is the vertex handler (Ch 33). Verification's contract with the rest of the node is simple and absolute: if it passes here, it broke no protocol rule; if it fails here, it is discarded and goes no further.

---

## 31.2 What it does and why it exists

Imagine the node had no verification stage and trusted whatever arrived. A malicious peer could send a "transaction" that spends coins it never owned (no valid signature), or that prints money out of thin air (outputs worth more than inputs), or a "block"

whose proof-of-work was never actually done. The node would store it, the indexes would record fake balances, and — worse — it would relay the garbage on to its peers. The whole network's promise ("trust no one, check everything") would collapse.

Verification exists so that **each vertex carries its own proof of legitimacy, and the node checks that proof in isolation.** "In isolation" is the key phrase. A verifier may need to read the inputs a transaction references (to see how much they were worth and what lock they carry), and it may need the parents (to check timestamps), but it does **not** ask "is this the best chain?" or "does this conflict with some other transaction the network prefers?" Those are global, comparative questions about which history wins. Verification only asks the local, absolute question: taken on its own terms, does this vertex break any rule?

That distinction maps onto a vocabulary you met in Chapter 10.

> **Recap — invalid vs. conflict (full treatment in Ch. 10 & 32).** A vertex is **invalid** when it breaks a protocol rule: bad signature, bad proof-of-work, money created from nothing, malformed structure. An invalid vertex is wrong — it can never be part of any valid history, so it is rejected and forgotten. A vertex is in **conflict** when it is perfectly valid on its own but competes with another valid vertex (two transactions spend the same coin). Conflicts are not errors; they are resolved by consensus, which marks the loser **voided** (kept, but flagged as not-counting) rather than deleting it. **Verification handles invalidity. Consensus handles conflict.** This chapter is entirely about the first; → Ch 32 for the second.

So the package's job, stated tightly: **given one vertex, run every applicable protocol rule against it, raising an exception the moment any rule is broken; raise nothing and the vertex is valid.** The verifiers do not return `True`/`False` per rule — they either return quietly or raise a specific exception (`PowError`, `InputOutputMismatch`, `InvalidInputData`, …). Verification is, in effect, a long list of assertions about correctness.

---

## 31.3 The concepts it rests on

Verification is almost pure consumer of ideas defined earlier in the book. It is where those ideas get enforced. Four recap boxes set the stage; the rest of the chapter shows the enforcement code.

**Recap — the vertex and its scripts (full treatment in Ch. 25).** Every block and transaction is a vertex. A `Transaction` has **inputs** (each naming an earlier output it wants to spend, `TxInput.tx_id` + `TxInput.index`, plus an `data` field holding an unlocking script) and **outputs** (each carrying a `value`, a `token_data` byte, and a `script` — the locking script). A `Block` has outputs (its reward) but no inputs. Verification reads these fields and checks the rules over them. → Ch 25 for the full data model.

**Recap — the UTXO model and value conservation (full treatment in Ch. 7).** Hathor tracks unspent transaction outputs (UTXOs), not account balances. A transaction consumes whole outputs as inputs and creates new outputs. The core monetary rule: for each token, the sum of inputs must equal the sum of outputs — coins are neither created nor destroyed — **unless** the transaction holds explicit mint or melt authority for that token. Ownership of an output is "whoever can satisfy its locking script." Verification is where that rule and that ownership check are actually performed. → Ch 7.

**Recap — proof-of-work, weight, target (full treatment in Ch. 9).** Each vertex carries a `weight` (a float; `weight = log2` of the expected number of hashing attempts). From the weight you derive a numeric **target**: `target = 2**(256 - weight) - 1` (`base_transaction.py:361`). Proof-of-work is valid when the vertex's own hash, read as a 256-bit integer, is **less than** that target — i.e. the miner found a hash small enough, which on average takes `2**weight` tries. Verification also checks that the weight is at least the network-required minimum, so a vertex can't claim a trivially-easy target. → Ch 9.

**Recap — validation state (full treatment in Ch. 25).** Each vertex's metadata records a `ValidationState`: `INITIAL` → `BASIC` → `FULL` (or the checkpoint variants), or `INVALID` (`transaction/validation_state.py:46–50`). It records how far verification has progressed. `BASIC` means "the checks that need no other data have passed." `FULL` means "every check, including those that read the inputs and parents, has passed." This state is what lets the node verify a vertex in two stages — central to §31.5.

---

## 31.4 The code, walked — the orchestrator and the verifier family

### 31.4.1 One service, many verifiers

The package follows a clean division of labour. There is **one orchestrator** — `VerificationService` (`verification_service.py:34`) — whose job is to call the right checks, in the right order, for whatever vertex type it is handed. The actual checks live in a **family of small verifier classes**, one per vertex type, each holding only the rules specific to that type:

- `VertexVerifier` (`vertex_verifier.py:52`) — rules shared by every vertex: proof-of-work, parents, outputs, sigops on outputs, headers, version.
- `BlockVerifier` (`block_verifier.py:35`) — block-only rules: reward amount, height, no inputs, checkpoints, mandatory feature signalling.
- `TransactionVerifier` (`transaction_verifier.py:67`) — tx-only rules: input scripts (signatures), value conservation, conflicts within the tx, the reward lock.
- `MergeMinedBlockVerifier`, `PoaBlockVerifier`, `TokenCreationTransactionVerifier`, plus the nano/fee/blueprint header verifiers — the niche types.

These eight are bundled into a single `NamedTuple`, `VertexVerifiers` (`vertex_verifiers.py:32`), built once at boot by `create_defaults` (`vertex_verifiers.py:43`) and handed to the service. The service reaches them as `self.verifiers.vertex`, `self.verifiers.block`, `self.verifiers.tx`, and so on. Why a `NamedTuple` and not a loose pile of constructor arguments? It is one object to pass around, every field is named and typed, and it is immutable — you cannot accidentally swap a verifier at runtime.

> **Recap — the facade pattern (full treatment in Ch. 3).** `VerificationService` is a facade: callers (the vertex handler) say "validate this vertex" and the service hides the messy choreography — which type it is, which verifiers to call, in which order, with which checks skipped. The caller never touches an individual verifier. This keeps the rule-ordering logic in exactly one place. → Ch 3 for the pattern.

### 31.4.2 Dispatch by vertex type

How does the service know which type-specific checks to run? It reads the vertex's `version` field and branches with a `match` statement. Here is the pattern, from `verify` (`verification_service.py:182`):

```python
match vertex.version:
    case TxVersion.REGULAR_BLOCK:
        assert type(vertex) is Block
        self._verify_block(vertex, params)
    case TxVersion.REGULAR_TRANSACTION:
        assert type(vertex) is Transaction
        self._verify_tx(vertex, params)
    case TxVersion.TOKEN_CREATION_TRANSACTION:
        ...
    case _:  # pragma: no cover
        assert_never(vertex.version)
```

Two details worth noticing, because they are deliberate. First, the `assert type(vertex) is Block` uses `type(...) is` rather than `isinstance(...)`; the comment at `verification_service.py:114` explains it — each subclass gets its own branch, so an exact-type check (not a subclass-accepting one) catches a vertex that lied about its version. Second, the final `case _: assert_never(...)` is a typing trick: `assert_never`[1] makes the type-checker prove that every `TxVersion` has a branch. If someone adds a new vertex version and forgets to handle it here, mypy fails the build. The dispatch is exhaustive by construction.

---

## 31.5 The basic-vs-full split

This is the single most structural idea in the package, so it earns its own section.

Some rules can be checked with **nothing but the vertex itself in hand**. Is the proof-of-work valid? Just hash the bytes and compare to the target — no other data needed. Are the outputs all positive and not too numerous? Just read the outputs. Is the weight a finite number? Just read the field.

Other rules **require the rest of the ledger**. To check a transaction's signatures, you must fetch the outputs it is spending — and those live in other transactions, which must already be in storage. To check value conservation, you must know how much each spent output was worth. To check parents' timestamps, you must fetch the parents.

Hathor names these two tiers **basic** and **full**, and `VerificationService` exposes them as two entry points:

```python
def validate_basic(self, vertex, params) -> bool:        # verification_service.py:50
    ...
    self.verify_basic(vertex, params)
    vertex.set_validation(ValidationState.BASIC)

def validate_full(self, vertex, params, ...) -> bool:    # verification_service.py:64
    ...
    self.verify(vertex, params)
    vertex.set_validation(ValidationState.FULL)
```

Note the naming convention, because it is easy to trip on: the `validate_*` methods (with the `d`) are the public entry points — they run the checks and update the vertex's validation state. The `verify_*` methods are the workers — they run the checks and raise on failure but **do not** touch the validation state (`verify_basic`'s docstring says so explicitly, `verification_service.py:107`). So `validate_basic` = `verify_basic` + record the new state.

The internal split runs all the way down. The pure-no-dependency checks are gathered under `verify_without_storage` (`verification_service.py:291`) — its very name advertises the contract: these checks read no other transaction. It is what `verify_basic` leans on, and it is also re-run as the first step of full verification. The dependency-needing checks live in `verify` (`verification_service.py:172`), which for a transaction calls `verify_inputs` (signatures, by reading the spent outputs), `verify_sum` (conservation), `verify_parents`, and the reward lock.

**Why split at all?** Because of sync order. When a node is catching up, it may receive a transaction before it has the transactions that transaction spends. It cannot run full verification yet — the dependencies are not on disk. But it can run basic verification immediately (the proof-of-work is checkable in isolation), mark the vertex `BASIC`, and store it. Later, once every dependency has reached `FULL`, the node circles back and promotes the vertex to `FULL`. The basic tier is, in effect, a cheap early filter: it rejects obvious garbage (failed PoW) the instant it arrives, without waiting for data that may be hours away.

Here is the basic tier for a regular transaction, so you can see exactly which checks are deemed "no dependencies needed" (`verification_service.py:156`):

```
def _verify_basic_tx(self, tx, params) -> None:
    if tx.is_genesis:
        return
    self.verifiers.tx.verify_parents_basic(tx)            # count parents, no fetch
    if self._settings.CONSENSUS_ALGORITHM.is_pow():
        self.verifiers.tx.verify_weight(tx)              # weight >= minimum
    self.verify_without_storage(tx, params)              # PoW, output rules, sigops...
```

`verify_parents_basic` (`transaction_verifier.py:81`) only counts parents and checks for duplicates — it does not fetch them, so it qualifies as basic. The deeper parent check, `verify_parents` (`vertex_verifier.py:65`), does fetch every parent (to compare timestamps and confirm the right kinds of parent), so it belongs to full verification.

> **Genesis is exempt.** Throughout the verifiers you will see early returns like `if tx.is_genesis: return` (e.g. `verification_service.py:158`, `:220`, `:259`). The genesis block and transactions are hard-coded into the settings and agreed by every node by definition — there is no earlier history to validate them against, so they are accepted axiomatically. → Ch 0 / Ch 22 for genesis.

---

## 31.6 Proof-of-work: hash below target

The first rule any vertex must pass — and the cheapest to check — is proof-of-work. It is in the no-storage tier (`verification_service.py:335` for blocks, `:348` for txs), and the check itself is six lines (`vertex_verifier.py:142`):

```
def verify_pow(self, vertex, *, override_weight=None) -> None:
    assert self._settings.CONSENSUS_ALGORITHM.is_pow()
    numeric_hash = int(vertex.hash_hex, vertex.HEX_BASE)     # hash as a 256-bit int
    minimum_target = vertex.get_target(override_weight)      # 2**(256-weight) - 1
    if numeric_hash >= minimum_target:
        raise PowError(...)
```

The logic is exactly the Chapter 9 rule. The vertex's hash is a 256-bit number (read from hex, `HEX_BASE = 16`, `base_transaction.py:159`). The target is derived from the vertex's claimed weight (`get_target`, `base_transaction.py:361`). A higher weight means a smaller target, which means fewer acceptable hashes, which means more work to find one. If the hash is **not** below the target, the miner did not do the work they claimed, and `PowError` is raised.

But `verify_pow` only checks the hash against the weight the vertex claims. What stops a cheater from claiming `weight = 1` (trivial to mine) on a real transaction? A separate check: **minimum weight**. For transactions, `verify_weight` (`transaction_verifier.py:93`) recomputes the minimum weight the difficulty algorithm requires for a tx of this size and amount, and rejects anything meaningfully below it:

```
def verify_weight(self, tx) -> None:
    min_tx_weight = self._daa.minimum_tx_weight(tx)
    ...
    if tx.weight < min_tx_weight - self._settings.WEIGHT_TOL:
        raise WeightError(...)
```

For blocks the analogue is `BlockVerifier.verify_weight` (`block_verifier.py:60`), which calls the difficulty-adjustment algorithm[2] to compute the required block weight from the recent block history. So PoW is two rules working together: you must do the work for the weight you claim (`verify_pow`) **and** the weight you claim must be at least what the network demands (`verify_weight`). One without the other would be a hole.

---

## 31.7 Value conservation and token rules

The monetary heart of transaction verification is `verify_sum` (`transaction_verifier.py:257`). It enforces the conservation rule from Chapter 7: for each token, money in must equal money out, unless authority says otherwise.

The work begins before `verify_sum`, with a `token_dict` — a summary, per token, of how much that token is being minted or melted by the transaction. The convention (documented at `transaction_verifier.py:271`) is `amount = outputs − inputs`, so a

positive amount means the tx is trying to create that token (mint) and a negative amount means destroy it (melt). `verify_sum` then walks each token and applies the rules:

- For any non-native token whose amount is nonzero, `_check_token_permissions` (`transaction_verifier.py:337`) demands the matching authority: melting needs melt authority (`ForbiddenMelt`), minting needs mint authority (`ForbiddenMint`). No authority, no creation or destruction.
- For the **native token HTR**, the rule is strict and absolute. HTR cannot be minted or melted by an ordinary transaction; the only legitimate sources of new HTR are block rewards and the deposit/withdraw mechanics of custom tokens. The code computes an `htr_expected_amount` from those mechanics and then asserts the tx's HTR delta matches exactly: a surplus raises `InputOutputMismatch` ("invalid surplus of HTR", `:310`), a deficit raises the same ("invalid deficit", `:329`), and the function ends on `assert htr_info.amount == htr_expected_amount` (`:335`). HTR is conserved to the satoshi.

Two supporting checks sit nearby. `verify_outputs` (`vertex_verifier.py:153`) rejects any output with a non-positive value (`InvalidOutputValue`) and forbids an authority output on the native token (`InvalidToken` — you cannot hold mint/melt authority over HTR, because HTR has no mint/melt). And `verify_output_token_indexes` (`transaction_verifier.py:247`) makes sure every output's `token_data` index points at a token actually listed in the transaction.

A small worked check. Suppose a transaction has two inputs worth 5 HTR and 3 HTR (8 in) and two outputs worth 6 HTR and 2 HTR (8 out). The HTR delta is `8 − 8 = 0`; `htr_expected_amount` is 0 (no token deposit/withdraw); `0 == 0`, conservation holds, accepted. Now suppose the outputs were 6 and 3 (9 out): delta `9 − 8 = +1`, a surplus of 1 HTR — money from nothing — and `verify_sum` raises `InputOutputMismatch`. That single comparison is what makes the currency real.

For **token-creation** transactions there is an extra layer: `TokenCreationTransactionVerifier` (`token_creation_transaction_verifier.py:24`) checks the new token's name and symbol are well-formed and that the version is permitted (`verify_token_info`, `:43`), and that the tx genuinely mints a positive amount of the new token (`verify_minted_tokens`, `:30`).

---

## 31.8 Script evaluation — the heart of signature checking

This is the deepest idea in the chapter, and the one that pays off Chapter 7's slogan: ownership is a satisfiable lock. We build it up carefully, because the mechanism — a tiny **stack machine** — may be new to you.

### 31.8.1 A stack machine, from scratch

A **stack**[3] is a list you only touch at one end: you push a value on top, and you pop the top value off. Last in, first out. A **stack machine** is a way of evaluating a program where every operation works by pushing and popping this single shared stack. There are no variables and no addresses — just the stack and a list of instructions.

Here is a neutral toy. Suppose our instructions are: numbers (which mean "push me") and the word `ADD` (which means "pop two numbers, push their sum"). Evaluate the program `3 4 ADD`:

```
instruction   action                       stack after
-----------   --------------------------   -----------
3             push 3                        [3]
4             push 4                        [3, 4]
ADD           pop 4, pop 3, push 3+4=7      [7]
```

The program ends with `[7]` on the stack — that is its result. Now add a verb `EQ` ("pop two, push 1 if equal else 0") and a rule: the program is valid only if it ends with exactly one value, and that value is 1 (true). Evaluate `5 5 EQ`:

```
5    push 5    [5]
5    push 5    [5, 5]
EQ   5==5 → 1  [1]
```

Ends with `[1]` — valid. The program `5 6 EQ` would end with `[0]` — invalid. That is the entire conceptual model: instructions push data or transform the top of the stack, and "success" is a single `1` left at the end. Hathor's script language is exactly this idea with cryptographic verbs added.

### 31.8.2 The locking/unlocking split

Recall from Chapter 7 that ownership is encoded as two scripts. The **output** carries a locking script (the puzzle: "to spend me, satisfy this"). The **input** that wants to spend it carries an unlocking script (the solution: "here is the proof"). To check that the spend is legitimate, the node **concatenates the unlocking script followed by the locking script and runs the whole thing on one stack.** If it ends with a single `1`, the spend is authorized.

In code this is `script_eval` (`transaction/scripts/execute.py:103`), which the transaction verifier calls once per input via `verify_script` (`transaction_verifier.py:172`):

```
script_eval(tx, input_tx, spent_tx, params.features.opcodes_version)
```

`script_eval` reaches into the spent transaction, grabs the locking script of the exact output being spent (`spent_tx.outputs[txin.index].script`), and hands it plus the input's unlocking data to `raw_script_eval` (`execute.py:125`), which does the concatenation:

```
full_data = input_data + output_script   # unlock, then lock
execute_eval(full_data, log, extras)
```

`execute_eval` (`execute.py:54`) is the loop. It walks the bytes; pushdata bytes go on the stack, opcodes call their handler (`execute_op_code`); at the end it calls `evaluate_final_stack` (`execute.py:86`), which enforces our toy's exact rule — **the stack must hold exactly one value, and that value must be `1`** — raising `FinalStackInvalid` otherwise. The toy from §31.8.1 is the real success condition.

### 31.8.3 The real script: P2PKH

The standard Hathor output script is **P2PKH** — pay to public key hash (`scripts/p2pkh.py:25`). Its locking script is a fixed sequence of opcodes (`create_output_script`, `:70`):

```
OP_DUP  OP_HASH160  <pubKeyHash>  OP_EQUALVERIFY  OP_CHECKSIG
```

And the matching unlocking data the spender supplies (`create_input_data`, `:94`) is just two pushed values:

```
<signature>  <pubKey>
```

In English the locking script says: "to spend this coin, present a public key that hashes to this stored hash, and a signature that key validates." That is the lock; only the holder of the matching private key can produce both halves. Let us trace the concatenated program `<sig> <pubKey> OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG` step by step on the stack:

```
instruction       action                                   stack after
---------------   --------------------------------------   ----------------------
<sig>             push the signature                        [sig]
<pubKey>          push the public key                       [sig, pubKey]
OP_DUP            duplicate top                             [sig, pubKey, pubKey]
OP_HASH160        pop pubKey, push SHA256→RIPEMD160 of it   [sig, pubKey, hash(pubKey)]
<pubKeyHash>      push the hash stored in the output        [sig, pubKey, hash(pubKey), stor
OP_EQUALVERIFY    pop two, fail unless equal               [sig, pubKey]
OP_CHECKSIG       pop pubKey & sig, verify; push 1 if valid  [1]
```

It ends with `[1]` — valid. Each opcode is one small function in `opcode.py`:

- `op_dup` (`opcode.py:166`) pushes a copy of the top item. (This is how the public key gets used twice — once to hash, once to verify.)

- `op_hash160` (`opcode.py:275`) pops the top, hashes it SHA-256 then RIPEMD-160, pushes the digest. This is how an address is derived from a public key.
- `op_equalverify` (`opcode.py:200`) pops two items, and raises `EqualVerifyFailed` unless they are equal. This is the check "the key you gave hashes to the address this coin was sent to" — i.e. you are spending the right person's coin.
- `op_checksig` (`opcode.py:238`) pops a public key and a signature, and verifies the signature against the transaction's signing data (`tx.get_sighash_all_data()`, `opcode.py:266`) using that key. Valid → push `1`; invalid → push `0`. This is the proof that the spender holds the private key.

So the two cryptographic facts — you control the right address (`OP_EQUALVERIFY`) and you can sign for it (`OP_CHECKSIG`) — are what "ownership" reduces to, exactly as Chapter 7 promised. There is no account, no balance, no list of who-owns-what; there is only a lock and the question of whether you can open it.

### 31.8.4 The opcode dispatch table

How does the evaluator turn an opcode byte into the right function? A dictionary, in `execute_op_code` (`opcode.py:635`):

```
opcode_fns = {
    Opcode.OP_DUP: op_dup,
    Opcode.OP_EQUAL: op_equal,
    Opcode.OP_EQUALVERIFY: op_equalverify,
    Opcode.OP_CHECKSIG: op_checksig,
    Opcode.OP_HASH160: op_hash160,
    ...
}
opcode_fn = opcode_fns.get(opcode)
if opcode_fn is None:
    raise ScriptError(f'unknown opcode: {opcode}')
opcode_fn(context)
```

> **Recap — dispatch table (full treatment in Ch. 4).** Instead of a long `if opcode == ... elif ...` chain, the evaluator maps each opcode to its handler in a dictionary and calls whatever it finds. Adding an opcode is adding a row. An unknown opcode falls through to `None` and is rejected — the script language is closed, only the listed verbs exist. → Ch 4 for dispatch.

Notice the `version` parameter: some opcodes (the `OP_DATA_*` family, `OP_CHECKDATASIG`) are only added to the table under `OpcodesVersion.V1` (`opcode.py:654`). This is how the protocol can retire or gate opcodes over time via feature activation (Ch 38) without breaking old vertices.

Two design choices to absorb. First, the language is **deliberately limited** — there are no loops and no jumps. A script is a straight-line list of instructions that runs once, top to bottom. That is on purpose: a script that cannot loop cannot run forever, so

verification has a bounded cost. (Hathor's general-purpose programmable money is nano-contracts, Ch 39, which run under an explicit resource meter precisely because they are expressive.) Second, the multisig case (`raw_script_eval`, `execute.py:128`) runs the evaluator twice — once to check the redeem script matches its hash, once to check the signatures satisfy it — because a single concatenated pass would leave leftover signatures on the stack and fail the "exactly one value" rule.

---

## 31.9 Bounding the work: sigops, sizes, counts

Verification must be cheap, because the node runs it on every vertex from every peer; an attacker who could make one vertex expensive to check could grind the node to a halt. So several rules cap the size of the work, independent of correctness:

- **Sigops.** Signature verification is the most expensive operation in a script. The node counts the signature operations across all outputs (`verify_sigops_output`, `vertex_verifier.py:182`) and all inputs (`verify_sigops_input`, `transaction_verifier.py:105`), and rejects a vertex whose totals exceed `MAX_TX_SIGOPS_OUTPUT` / `MAX_TX_SIGOPS_INPUT` with `TooManySigOps`. A `SigopCounter` walks the script bytes counting `OP_CHECKSIG`-class opcodes without actually executing them.
- **Counts.** At most `MAX_NUM_INPUTS` inputs and `MAX_NUM_OUTPUTS` outputs (`verify_number_of_inputs`, `transaction_verifier.py:237`; `verify_number_of_outputs`, `vertex_verifier.py:177`), raising `TooManyInputs` / `TooManyOutputs`.
- **Sizes.** Each input's unlocking data is capped at `MAX_INPUT_DATA_SIZE` (`transaction_verifier.py:146`) and each output script at `MAX_OUTPUT_SCRIPT_SIZE` (`vertex_verifier.py:172`).

None of these are correctness rules in the "is this fraud?" sense. They are denial-of-service defenses: a bound on how much computation one vertex can demand.

---

## 31.10 Structural rules: parents, timestamps, no double-spend within a tx

Beyond money and signatures, a vertex must be structurally sound.

**Parents.** `verify_parents` (`vertex_verifier.py:65`) is the structural workhorse of full verification. It fetches every parent from storage (raising `ParentDoesNotExist` if one is missing), and checks: no duplicate parents (`DuplicatedParents`); each parent's timestamp is strictly before the vertex's (`TimestampError` — a vertex cannot confirm something from its own future); and the right shape of parents — a transaction must

have exactly 2 transaction parents and 0 block parents, a block exactly 2 transaction parents and 1 block parent (the constants at `vertex_verifier.py:42–47`), with blocks ordered before transactions, else `IncorrectParents`.

> **Recap — parents vs. inputs (full treatment in Ch. 8 & 25).** A vertex has two kinds of edge. **Parents** are the DAG confirmation links ("I build on these vertices"); **inputs** are the spending links ("I consume these outputs"). `verify_parents` checks the parent edges; `verify_inputs` checks the input edges. They are entirely separate checks over entirely separate fields. → Ch 8.

**No double-spend within a single transaction.** A transaction must not list the same output twice in its own inputs. `_verify_inputs` (`transaction_verifier.py:135`) tracks a `set` of `(tx_id, index)` keys and raises `ConflictingInputs` on a repeat. Note the scope: this catches a tx spending its own coin twice. The cross-transaction double-spend — two different txs spending the same coin — is a **conflict**, not an invalidity, and is left to consensus (Ch 32). `verify_conflict` (`transaction_verifier.py:389`) does reject one narrow case here: spending an output that is already spent by a confirmed (block-included, non-voided) transaction (`ConflictWithConfirmedTxError`) — because that competitor has already won, so the newcomer cannot be valid.

**Reward lock.** Freshly-mined block rewards cannot be spent immediately; they must "mature" for a number of blocks. `verify_reward_locked` (`transaction_verifier.py:191`) checks that any block reward this transaction spends is old enough relative to the best block height, raising `RewardLocked` otherwise. This prevents a miner from spending a reward that a reorg might later erase. (For blocks, the analogous maturity is checked via `verify_height`, `block_verifier.py:53`.)

---

## 31.11 Block-only rules

A block is not a transaction, and `BlockVerifier` (`block_verifier.py:35`) carries the rules that only make sense for blocks:

- **Reward amount** (`verify_reward`, `:69`): the sum of a block's outputs must equal exactly the protocol-defined issuance for its height (`get_tokens_issued_per_block`), else `InvalidBlockReward`. A miner cannot pay themselves more than the schedule allows. This is the one place HTR is legitimately created.
- **No inputs** (`verify_no_inputs`, `:79`): a block has no inputs; any input raises `BlockWithInputs`.
- **No custom tokens in outputs** (`verify_output_token_indexes`, `:84`): block reward outputs must be HTR only; `BlockWithTokensError` otherwise.

- **Data size** (`verify_data`, `:89`): the block's free-form data field is capped at `BLOCK_DATA_MAX_SIZE`.
- **Checkpoints** (`verify_checkpoints`, `:107`): a block must not fork the chain below a configured checkpoint height — `CheckpointError`. This is the hard backstop against deep reorgs (Ch 10).
- **Mandatory signalling** (`verify_mandatory_signaling`, `:93`): during a feature's `MUST_SIGNAL` phase, blocks are required to signal support; a silent block raises `BlockMustSignalError` (Ch 38).

The two specialized block types add one rule each: `MergeMinedBlockVerifier.verify_aux_pow` (`merge_mined_block_verifier.py:28`) checks the Bitcoin auxiliary proof-of-work and its Merkle path[4], and `PoaBlockVerifier.verify_poa` (`poa_block_verifier.py:28`) checks the authority signature and turn-based weight on proof-of-authority[5] networks — where there is no mining, so the block is signed by an authorized producer rather than mined.

---

## 31.12 How it plugs into the lifecycle

Verification does not run itself. It is invoked by the **vertex handler** (Ch 33), the component that owns the ingestion pipeline. On every new vertex — whether it arrived from a peer, was relayed, or was pushed via the API — the handler builds a `VerificationParams` (the per-run flags: which features are active, whether to reject stale or conflicting vertices, the opcode version) and calls `validate_full` (`vertex_handler/vertex_handler.py:213`):

```
if not metadata.validation.is_fully_connected():
    try:
        self._verification_service.validate_full(vertex, params)
    except HathorError as e:
        raise InvalidNewTransaction(f'full validation failed: {str(e)}') from e
```

The shape of the contract is right there. Verification raises on any broken rule; the handler catches it and turns it into `InvalidNewTransaction` — the vertex is rejected, never stored as valid, never passed to consensus, never relayed. If `validate_full` returns without raising, the vertex is individually valid, its state is now `FULL`, and the handler proceeds to the next stage — consensus — which decides where this now-trusted vertex sits in canonical history.

`VerificationParams` (`verification_params.py:24`) is worth one sentence: it is a frozen dataclass of per-run knobs, with a `default_for_mempool` constructor (`:38`) that sets the strict combination used for real-time vertices (reject too-old vertices, harden token restrictions, reject conflicts with confirmed txs). Different ingestion paths — syncing old history vs. accepting a fresh mempool tx — pass different params, which is how the same verifier code enforces slightly different policies depending on context.

## Recap

| Rule (what is checked) | Verifier method | File:line |
|---|---|---|
| Orchestration / dispatch by type | `VerificationService.verify` / `verify_basic` | `verification_service.py:172` / `:100` |
| Basic vs full split (public) | `validate_basic` / `validate_full` | `verification_service.py:50` / `:64` |
| No-dependency checks bundle | `verify_without_storage` | `verification_service.py:291` |
| Proof-of-work (hash < target) | `VertexVerifier.verify_pow` | `vertex_verifier.py:142` |
| Target from weight | `BaseTransaction.get_target` | `base_transaction.py:361` |
| Minimum weight (tx / block) | `verify_weight` | `transaction_verifier.py:93` / `block_verifier.py:60` |
| Value conservation + token authority | `TransactionVerifier.verify_sum` | `transaction_verifier.py:257` |
| Outputs positive, no HTR authority | `VertexVerifier.verify_outputs` | `vertex_verifier.py:153` |
| Input scripts (signatures) | `verify_inputs` → `verify_script` → `script_eval` | `transaction_verifier.py:131` / `:172` / `execute.py:103` |
| Script evaluation loop | `execute_eval` / `evaluate_final_stack` | `execute.py:54` / `:86` |
| Opcode dispatch | `execute_op_code` | `opcode.py:635` |
| P2PKH lock | `P2PKH.create_output_script` | `p2pkh.py:70` |
| Sigops bound | `verify_sigops_output` / `verify_sigops_input` | `vertex_verifier.py:182` / `transaction_verifier.py:105` |

| Rule (what is checked) | Verifier method | File:line |
|---|---|---|
| Parents (count, type, timestamp) | `VertexVerifier.verify_parents` | `vertex_verifier.py:65` |
| No double-spend within a tx | `TransactionVerifier._verify_inputs` | `transaction_verifier.py:135` |
| Reward lock (maturity) | `verify_reward_locked` | `transaction_verifier.py:191` |
| Block reward amount | `BlockVerifier.verify_reward` | `block_verifier.py:69` |
| Block has no inputs / no tokens | `verify_no_inputs` / `verify_output_token_indexes` | `block_verifier.py:79` / `:84` |
| Checkpoint forbids deep fork | `BlockVerifier.verify_checkpoints` | `block_verifier.py:107` |
| Mandatory feature signalling | `verify_mandatory_signaling` | `block_verifier.py:93` |
| Invoked by the vertex handler | `validate_full` call site | `vertex_handler.py:213` |

Verification is the node's lie detector. Every rule in this chapter is an absolute statement about one vertex — its work was done, its money balances, its signatures hold, its structure is legal — checked without reference to any competing history. That is precisely its boundary. A vertex can pass every check here and still not make it into the ledger that the network keeps, because two perfectly-valid transactions might spend the same coin. Deciding which of two valid-but-conflicting histories wins — comparing accumulated weight, voiding the loser, handling reorgs — is the next stage. Once a vertex is individually valid, the question becomes: which history does it belong to? That is Chapter 32, consensus.

---

1. `assert_never(x)` is a typing helper that the static type-checker treats as "this line must be unreachable." If the checker can prove some case reaches it (e.g. an unhandled enum value), the build fails. At runtime it raises if ever actually hit. It is how a `match` over a fixed set of cases is made provably exhaustive. ↩
2. **DAA** = Difficulty Adjustment Algorithm. The code that decides how hard mining should be right now, so blocks keep arriving at a steady average rate as total network hashing power rises and falls. Full treatment in Ch 32 / see Ch 9. ↩
3. A stack is a last-in-first-out collection: you can only add (push) to the top and remove (pop) from the top. Think of a stack of plates. A stack machine evaluates a program using only one such stack as its working memory. ↩
4. A Merkle path is a short list of hashes that proves one item belongs to a larger hashed set without revealing the whole set. Merged mining uses it to prove a Hathor block was committed inside a Bitcoin block. → Ch 37. ↩
5. Proof-of-authority (PoA) is a consensus variant for private networks where blocks are not mined but signed by a fixed set of authorized producers taking turns. There is no proof-of-work and no mining reward. → Ch 32. ↩